

文化祭プロジェクト 仕組みドキュメント

AI パーソナリティ診断

アプリの説明と仕組みガイド

IT 系専門学校 1 年生向け | チームメンバー共有資料

目的	ユーザーの行動パターンから AI が 8 つのテック企業タイプを診断
技術	FastAPI (Python) / Vanilla JS / OpenRouter AI API
特徴	質問は毎回 AI が動的生成 / 4 言語対応 / 無料・登録不要
デプロイ	Render → Railway に移行済み

<https://ai-personality-app-production.up.railway.app/>

1. アプリ全体像

このアプリは「日常の行動パターンに関する質問に答えると、あなたが Apple・Google など 8 つのアメリカテック企業のどのタイプかを AI が診断する」という文化祭向け体験アプリです。

ユーザーの体験フロー



8 つのパーソナリティタイプ

Apple 透明感・洗練・静かな余裕	Google 好奇心・明るさ・遊び心	Amazon 行動力・実用感・テンポの良さ	Microsoft 安心感・やさしさ・誠実さ
Tesla 情熱・強い引力・勢い	Meta 共感・つながり・会話力	Nvidia ミステリアス・深夜感・静かな引力	Netflix 映画感・余韻・感情表現の強さ

ポイント: 質問文には企業名や製品名は一切登場しない。行動の癖や選択パターンから判定するので回答者は診断が終わるまでどのタイプになるかわからない。これが「当たった感」を生む。

2. 技術スタック（使っている技術）

バックエンド

FastAPI (Python)

サーバー側の処理を担当。ブラウザからのリクエストを受け取り、AIに問い合わせで結果を返す。
「FastAPI」はPythonで作られた高速なWebフレームワーク。

フロントエンド

HTML / CSS / Vanilla JS

画面表示を担当。ReactなどのフレームワークはあえPC使わず、シンプルなHTML+JSで構築。
アニメーションやAPI通信もピュアなJavaScriptで実装。

AI

OpenRouter API (Gemini 2.5 Flash)

Googleが開発した大規模言語モデル（LLM）をAPI経由で利用。
質問の生成・診断結果の文章作成の両方に使用。

デブロイ

Railway

アプリをインターネット上に公開するサービス。コードをpushするだけで自動でサーバーが立ち上がる。
もともとRenderで開発し、Railwayに移行済み。

ファイル構成

```
ai-personality-app/  
├── backend/  
│   └── main.py    ← サーバーの全ロジック（FastAPI）  
├── frontend/  
│   ├── index.html ← 画面のHTML  
│   ├── style.css  ← デザイン  
│   └── script.js   ← 画面の動き・API通信  
└── requirements.txt ← 必要なPythonライブラリ一覧
```

通信の仕組み（クライアント ↔ サーバー）

GET /api/questions

AIが生成した10問の質問リストを返す

POST /api/diagnosis

ユーザーの回答を受け取りAIで診断して結果を返す

GET /

index.html（画面）を返す

3. 質問生成のロジック

ユーザーがアプリを開くと、まずAIに「10問の質問を生成して」と依頼します。ここが最も工夫されている部分です。

なぜ毎回 AI が質問を生成するのか？

- 固定の質問だと、2回目以降に「答えを覚えて操作」できてしまうため
- 毎回違う質問にすることで、診断に新鮮感・驚きが生まれる
- AIが状況や雰囲気を変えながら生成するため、自然な日本語の質問になる

質問生成のしくみ（ステップ）

Step 1 タイプのレイアウトを決める

10問×4択で合計40個の選択肢それぞれに、8タイプのうちどれかを割り当てる。
例: Q1の4択は [Apple, Google, Amazon, Microsoft]、Q2は [Tesla, Meta, Nvidia, Netflix] など。
毎回ランダムに組み合わせを変えている。

Step 2 カテゴリ・シーン・文体をランダム選択

social（友達との会話）/school（学校生活）/solo（一人の時間）など5カテゴリからランダムに10テーマを選ぶ。さらに「帰り道」「雨の日」などの場面と「もし～なら」「あなたは今～」などの文章スタイルも毎回変える。

Step 3 AIへの指示（プロンプト）を組み立てる

ステップ1・2の情報を使って「こういうルールで10問作って」という指示文を作る。
禁止ワードも指定（企業名・恋愛・ガジェットなど）。

Step 4 OpenRouter API に送信して質問を受け取る

AIからJSON形式で質問データが返ってくる。
JSONの形式が正しいか・禁止ワードがないかをチェックしてからユーザーに渡す。
失敗した場合は最大3回まで自動でリトライする。

キャッシュ機能: 質問はバックグラウンドで先読み生成されており、ユーザーを待たせない工夫がされている。
同じユーザーが連続アクセスした場合はキャッシュを利用。

4. 診断結果のロジック

10問に答えたデータをサーバーに送ると、どのタイプかを判定して診断結果を生成します。

タイプ判定のしくみ

各回答には選んだ時点でタイプ（例: Apple）が紐づいている。

10問の回答を集計して、最も多く選ばれたタイプが「メインタイプ」になる。

2番目に多いタイプは「サブタイプ」として診断の深みに使われる。

例: 10問の回答集計結果



AI が生成する診断内容

タイプが決まったら、AIに「このタイプのパーソナリティ診断結果を書いて」と依頼します。

- **診断コメント** そのタイプらしい個性・特徴の説明文（160～200文字）
- **特性チャート** 「好奇心」「安定感」など4項目を★1～5で評価
- **内面タグ** 「こだわり派」「直感型」など3つのキーワード
- **相性タイプ** 相性の良い企業タイプ上位3つと理由

フォールバック（AIが失敗した場合の保険）

AIの返答が不正だったり時間がかかりすぎた場合は、あらかじめコードに書いておいた

固定データ（fallback）を使って結果を表示する。ユーザーをエラー画面に飛ばさない設計。

- AI成功時:** AIが生成した個性豊かなオリジナルの診断コメントを表示
- AI部分失敗:** フィールド単位で確認し、途中切れなど不正な値だけ固定テキストに差し替え（他はAI結果を使用）
- AI完全失敗:** 全フィールドをコードに書いた固定データで代替表示。ユーザーにはエラーを見せない

5. 工夫ポイントと技術的な見どころ

◆ 診断結果でテーマカラーが変わる

診断結果のタイプ（Apple・Googleなど）に応じて、画面全体の配色が自動で切り替わる。各タイプに独自のカラーパレットが定義されており、視覚的に「自分のタイプ」を演出できる。

◆ 言語設定の永続化

ユーザーが選んだ言語（日本語・英語など）をブラウザのlocalStorageに保存する。次回アクセス時も前回と同じ言語で自動表示されるため、設定し直す手間がない。

◆ サブタイプの活用

10問の回答で2番目に多かったタイプが「サブタイプ」になる。相性診断の結果にサブタイプが優先的に登場するなど、診断の深みを出すために活用される。

◆ 4言語対応

日本語・英語・中国語・韓国語に対応。画面右上のボタンで切り替え可能。質問文も診断結果もすべての言語でAIが生成するため、完全なローカライズを実現。

◆ プロンプトエンジニアリング

AIへの指示文（プロンプト）を丁寧に設計することがこのアプリの核心。「企業名を出すな」「恋愛に触れるな」「毎回違う雰囲気にしろ」などの細かいルールをプロンプトに埋め込んでいる。

◆ キャッシュ & スレッド処理

質問生成はバックグラウンドスレッドで先行実行し、ユーザーの待ち時間をゼロに近づける。Pythonのthreadingモジュールを使用。

◆ 入力バリデーション

AIの返答が正しいJSON形式かどうか、禁止タイプが含まれていないかをチェック。不正な値が1つでもあればfallbackに切り替える堅牢な設計。

用語まとめ（わからない言葉リスト）

API	Application Programming Interface の略。 別のサービスの機能を自分のアプリから呼び出す仕組み。
LLM	Large Language Model（大規模言語モデル）の略。 ChatGPT や Gemini のような「大量のテキストで学習した AI」のこと。
JSON	JavaScript Object Notation の略。データをやり取りする際の標準フォーマット。 { "key": "value" } の形で書く。
FastAPI	PythonでWebサーバーを作るためのフレームワーク（枠組み）。 Djangoより軽量で高速。
プロンプト	AIへの指示文のこと。どう書くかで AIの出力品質が大きく変わる。 このアプリの質問生成・結果生成はどちらもプロンプトが要。
デプロイ	アプリを自分のPCからインターネット上のサーバーに載せて 誰でもアクセスできる状態にすること。

6. まとめ — このアプリで使った技術の全体像

フロントエンド HTML / CSS / JavaScript

ユーザーが見る画面。ボタン操作・アニメーション・APIへのリクエスト送信を担当。

バックエンド Python + FastAPI

サーバーの脳みそ。質問生成・診断判定・AI通信・フォールバック処理をすべて管理。

AI 活用 OpenRouter → Gemini 2.5 Flash

質問文の自動生成と診断コメントの作成。プロンプト設計がアプリの品質を左右する。

デプロイ Railway (クラウド)

コードを push するだけで自動でサーバーが更新される CI/CD 環境。

多言語 ja / en / zh / ko の 4 言語

言語ごとにプロンプトと出力ルールを切り替え。外国人来場者も利用可能。

このアプリで学べること

- APIを使ったAIの活用方法（プロンプトエンジニアリング）
- フロントエンドとバックエンドの役割分担と通信の仕組み
- Pythonによるサーバーサイド開発の基礎（FastAPI）
- クラウドへのデプロイの流れ
- ユーザー体験を考えた設計（フォールバック・キャッシュ・多言語化）

アプリはこちらから体験できます

<https://ai-personality-app-production.up.railway.app/>